Parallel & Distributed Computing
Lecture Week 4:
Parallel Programming Models

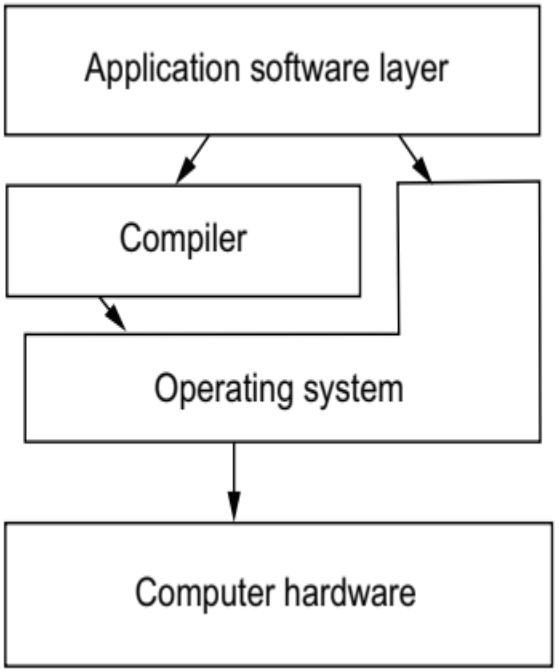
Farhad M. Riaz

Farhad.Muhammad@numl.edu.pk

Department of Computer Science
NUML, Islamabad

How does parallel computing works?

- As a developer, you are responsible for the application software layer, which includes your source code.
- In the source code, you make choices about the programming language and parallel software interfaces you use to leverage the underlying hardware.
- Additionally, you decide how to break up your work into parallel units.
- Approaches a developer can take into
  - Process-based parallelization
  - Thread-based parallelization
  - Vectorization
  - Stream (GPU) processing



Example for Sample Application

- Perform the computation on a regular two-dimensional (2D) grid of rectangular elements or cells.
- The steps prepare for the calculation are
  - Discretize (break up) the problem into smaller cells or elements
  - Define a computational kernel (operation) to conduct on each element
  - Add the following layers of parallelization on CPUs and GPUs to perform the calculation:
    - Vectorization
    - Threads
    - Processes
    - Off-loading the calculation to GPUs

Complexity

- In general, parallel applications are much more complex than corresponding serial applications.
- Not only do you have multiple instruction streams executing at the same time, but you also have data flowing between them.
- The costs of complexity are measured in programmer time in virtually every aspect of the software development cycle:
  - Design
  - Coding
  - Debugging
  - Tuning
  - Maintenance
- Adhering to "good" software development practices is essential when working with parallel applications - especially if somebody besides you will have to work with the software.

Portability

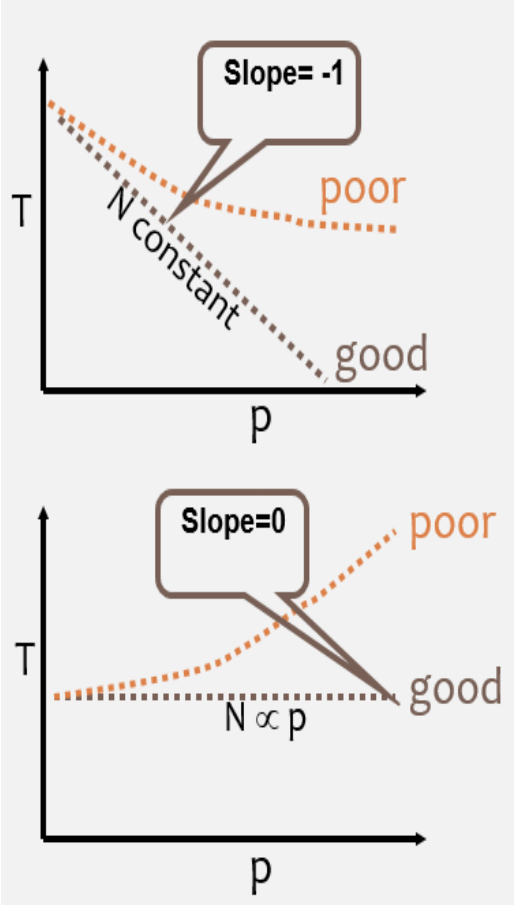
- Thanks to standardization in several APIs, such as MPI, POSIX threads, and OpenMP, portability issues with parallel programs are not as serious as in years past.
- All of the usual portability issues associated with serial programs apply to parallel programs. For example, if you use vendor "enhancements" to Fortran, C or C++, portability will be a problem.
- Even though standards exist for several APIs, implementations will differ in a number of details, sometimes to the point of requiring code modifications in order to effect portability.
- Operating systems can play a key role in code portability issues.
- Hardware architectures are characteristically highly variable and can affect portability.

Resource Requirements

- The primary intent of parallel programming is to decrease execution wall clock time, however in order to accomplish this, more CPU time is required. For example, a parallel code that runs in 1 hour on 8 processors actually uses 8 hours of CPU time.
- The amount of memory required can be greater for parallel codes than serial codes, due to the need to replicate data and for overheads associated with parallel support libraries and subsystems.
- For short running parallel programs, there can actually be a decrease in performance compared to a similar serial implementation. The overhead costs associated with setting up the parallel environment, task creation, communications and task termination can comprise a significant portion of the total execution time for short runs.

Scalability

- Two types of scaling based on time to solution: strong scaling and weak scaling.
- Strong scaling:
  - The total problem size stays fixed as more processors are added.
  - Goal is to run the same problem size faster
  - Perfect scaling means problem is solved in 1/P time (compared to serial)
- Weak scaling:
  - The problem size per processor stays fixed as more processors are added. The total problem size is proportional to the number of processors used.
  - Goal is to run larger problem in same amount of time
  - Perfect scaling means problem Px runs in same time as single processor run



Parallel Computers

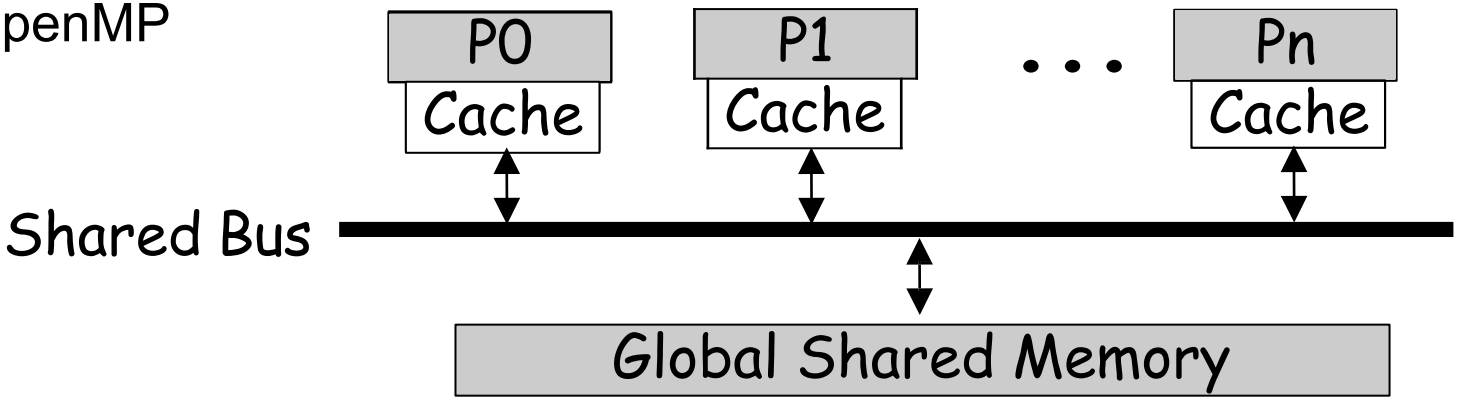
- Programming mode types
  - Shared Memory
  - Distributed Memory
  - Hybrid Model

Parallel Programing Models

- Parallel programming models exist as an abstraction above hardware and memory architectures
- These models are NOT specific to a particular type of machine or memory architecture
- These models can (theoretically) be implemented on any underlying hardware
- Examples from past
  - **SHARED memory model on a DISTRIBUTED memory machine**. Kendall Square Research (KSR) ALLCACHE approach, “virtual shared memory"
  - **DISTRIBUTED memory model on a SHARED memory machine**. Message Passing Interface (MPI) on SGI Origin 2000, employed the CC-NUMA type of shared memory architecture, however, MPI commonly done over a network of distributed memory machines
- **Which model to use?**
  - Combination of what is available and personal choice

Shared Memory Architecture

- Processors have direct access to global memory and I/O through bus or fast switching network
- Cache Coherency Protocol guarantees consistency of memory and I/O accesses
- Each processor also has its own memory (cache)
- Data structures are shared in global address space
- Concurrent access to shared memory must be coordinated
- Programming Models
  - Multithreading (Thread Libraries)
  - OpenMP



Threads Model

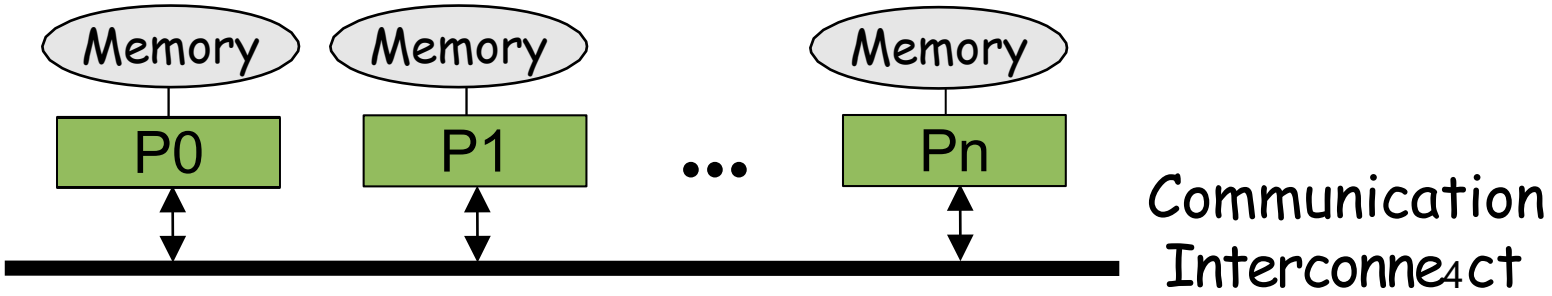
- Threads implementations commonly comprise:
  - A library of subroutines that are called from within parallel source code
  - A set of compiler directives imbedded in either serial or parallel source code
- Historically, hardware vendors have implemented their own proprietary versions of threads, making it difficult for programmers to develop portable threaded applications
- Standardization efforts: POSIX Threads (IEEE POSIX 1003.1c) and OpenMP (Industry standard)
  - POSIX Part of Unix/Linux, Library based
  - OpenMP Compiler directive based, Portable / multi-platform
  - Mircosoft threads, Java, Python threads, CUDA threads for GPUs

OpenMP

- OpenMP: portable shared memory parallelism
- Higher-level API for writing portable multithreaded applications
- Provides a set of compiler directives and library routines for parallel application programmers
- API bindings for Fortran, C, and C++

Distributed Memory Architecture

- Each Processor has direct access only to its local memory
- Processors are connected via high-speed interconnect
- Data structures must be distributed
- Data exchange is done via explicit processor-to-processor communication: send/receive messages
- Programming Models
  - Widely used standard: MPI
  - Others: PVM, Express, P4, Chameleon, PARMACS, ...



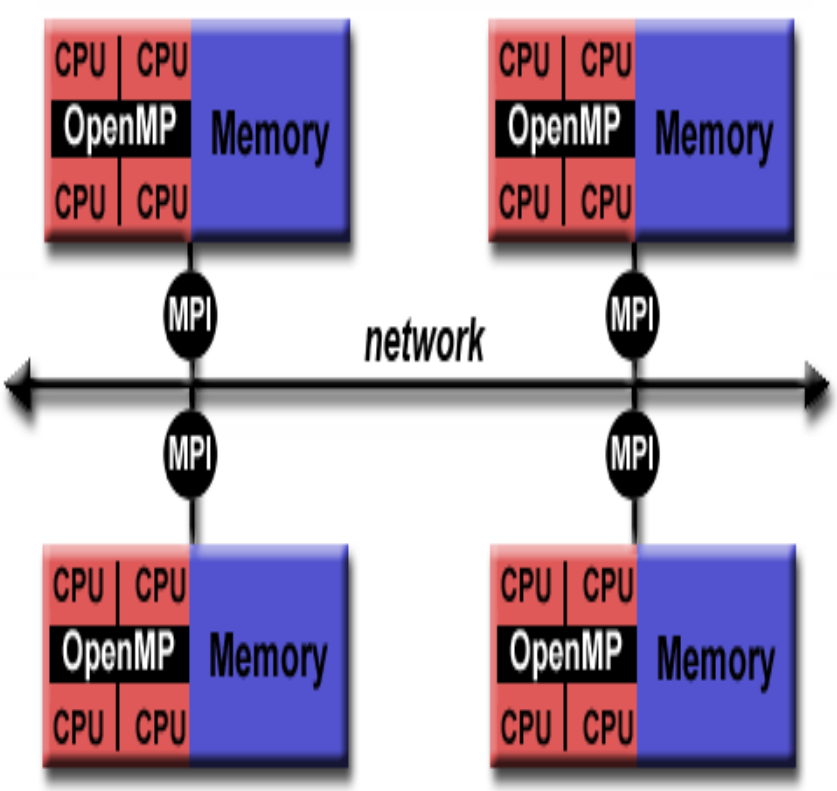
Message Passing Interface

MPI provides:

- Point-to-point communication
- Collective operations
  - Barrier synchronization
  - gather/scatter operations
  - Broadcast, reductions
- Different communication modes
  - Synchronous/asynchronous
  - Blocking/non-blocking
  - Buffered/unbuffered
- Predefined and derived datatypes
- Virtual topologies
- Parallel I/O (MPI 2)
- C/C++ and Fortran bindings

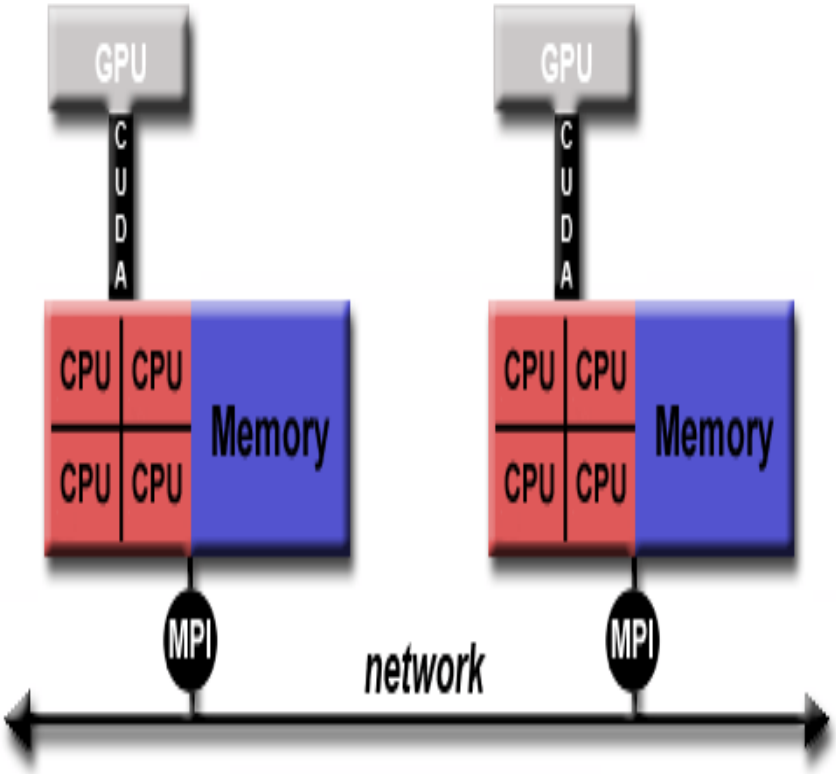
Hybrid Model

- A hybrid model combines more than one of the previously described programming models.
- A common example of a hybrid model is the combination of the message passing model (MPI) with the threads model (OpenMP).
- Threads perform computationally intensive kernels using local, on-node data
- Communications between processes on different nodes occurs over the network using MPI



Hybrid Model

- Another similar and increasingly popular example of a hybrid model is using MPI with CPU-GPU (Graphics Processing Unit) programming.
- MPI tasks run on CPUs using local memory and communicating with each other over a network.
- Computationally intensive kernels are off-loaded to GPUs onnode.
- Data exchange between node-local memory and GPUs uses CUDA (or something equivalent)

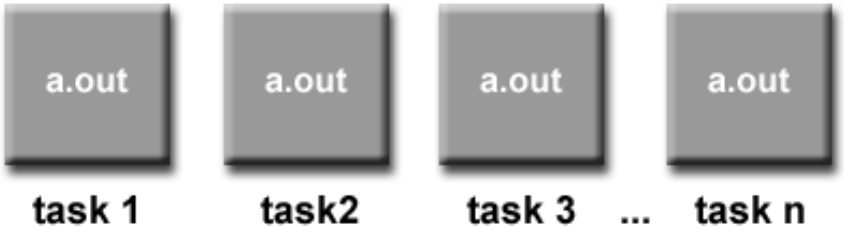


High level programming model

- Single Program Multiple Data (SPMD)
- Multiple Program Multiple Data (MPMD)

SPMD

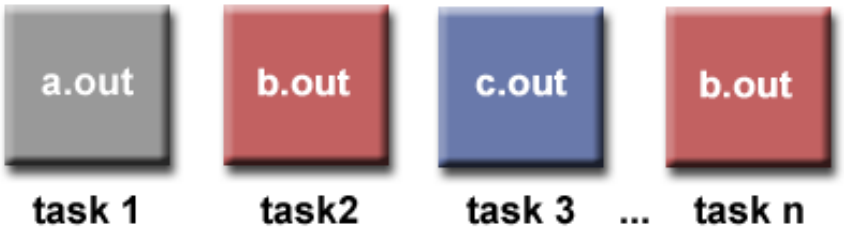
- Built upon any combination of the previously mentioned parallel programming models
- **SINGLE PROGRAM:** All tasks execute their copy of the same program simultaneously. This program can be threads, message passing, data parallel or hybrid.
- **MULTIPLE DATA**: All tasks may use different data



-tasks do not necessarily have to execute the entire program
- perhaps only a portion of it

MPMD

- built upon any combination of the previously mentioned parallel programming models
- **MULTIPLE PROGRAM:** Tasks may execute different programs simultaneously.
- The programs can be threads, message passing, data parallel or hybrid.
- **MULTIPLE DATA:** All tasks may use different data



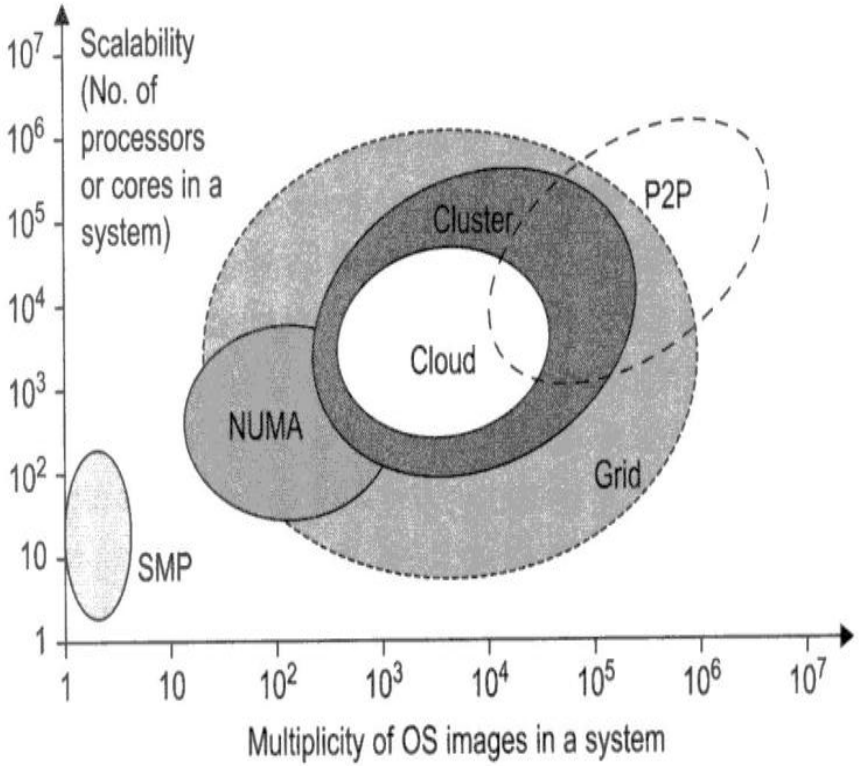
MPMD applications are not as common as SPMD applications

Parallel and Distributed Programming Models

- OPENMP
- MPI
  - For message passing systems
- MapReduce and BigTable
  - For internet clouds and data centers
- Service clouds require extension of Hadoop, EC2, S3 to facilitate distributed computing over distributed storage system
- CUDA
  - For NVIDIA GPUs
- Open Grid Service Architecture (OGSA)
  - For grid application development

Performance, Security and Energy Efficiency

- Performance Metrics
  - CPU Speed, FLOPS, Job response time, network latency, system throughput, network bandwidth, System overhead (OS boot time, compile time, etc).
- Scalability
  - Machine (size), software, application, and technology scalability
- Amdahl's law



Performance, Security and Energy Efficiency

- Security
  - Threats to system and network
  - Confidentiality, integrity, and availability
  - Copyright protection
  - System Defense technologies
  - Data protection infrastructures (IDS)
- Energy efficiency
  - Distributed power management
  - Unused servers' energy consumption
  - Reducing energy in active servers

That's all for today!!